

Automated AP Invoice Processor

OCR Engine & Document Intelligence Pipeline — Project Overview

Prepared by: **Makky**

Date: **3 July 2026**

Audience: **Engineering Team & Finance Leadership**

Status: **Internal Draft — For Review**

Document Scope

This brief covers what the Automated AP Invoice Processor is, the specific pain points it addresses, the limitations of traditional OCR approaches, and the technical solutions deployed to overcome them. It is intended to align both engineering and finance stakeholders on the project direction and rationale.

SECTION 1

What We Are Building

The Automated AP Invoice Processor is a **document intelligence pipeline** that ingests Accounts Payable invoices and automatically extracts, validates, and exposes structured financial data for downstream processing — eliminating manual data entry and reducing the risk of payment errors.

Pipeline Architecture

The system is composed of four tightly integrated layers, from raw document ingestion through to human review:

Layer	Component	Responsibility
1 — OCR Parsing	GCP Document AI	Ingests invoice files (PDF, image) and performs raw optical character recognition, producing structured document text with positional awareness.
2 — LLM Extraction	Large Language Model Layer	Interprets Document AI output and returns a clean, structured JSON payload containing all key invoice fields.
3 — Validation Engine	Automated Validator	Sums all line item amounts and compares the result to the declared invoice total, flagging any discrepancy for review.
4 — Exposure Layer	FastAPI + Streamlit	Exposes the pipeline via a REST API for system integration and a Streamlit UI for human-in-the-loop review and exception handling.

Structured Output Schema

Every processed invoice produces a standardised JSON object. This schema is the single source of truth passed to downstream ERP systems, accounting software, and reporting tools:

Field	Type	Description
vendor	string	The name of the supplying vendor as it appears on the invoice.

Field	Type	Description
invoice_date	string (ISO 8601)	The date the invoice was issued, normalised to a consistent date format.
total	decimal	The declared invoice total as stated by the vendor.
line_items[]	array of objects	One entry per line item on the invoice. Each object contains: description, quantity, unit_price, and amount.

Human Review Interface

A **Streamlit UI** provides finance teams with a clear view of processed invoices, flagged discrepancies, and validation outcomes. This keeps skilled staff engaged on exceptions and edge cases — not on routine data entry.

SECTION 2

The Pain Points We Are Solving

Accounts Payable operations at most organisations are burdened by a set of well-understood but persistently unresolved inefficiencies. This project directly targets five of them.

1 — Manual Data Entry

Finance staff are currently required to hand-key vendor names, invoice dates, totals, and individual line items into ERP systems for every invoice received. This process is **slow, expensive, and error-prone** — each manual keystroke is an opportunity for a transcription mistake, and the cumulative labour cost across high invoice volumes is substantial. The problem scales directly with transaction volume, meaning that business growth amplifies the inefficiency rather than reducing it.

2 — No Validation Layer

Invoice totals are rarely cross-checked against individual line items during manual processing. As a result, discrepancies between declared totals and summed line amounts routinely go undetected until they surface as **payment errors or audit failures**. There is no systematic mechanism to catch these discrepancies at the point of ingestion — the gap is structural, not procedural.

3 — Poor Searchability

Physical and scanned invoices are stored in unstructured folder systems — on shared drives, email inboxes, or physical filing cabinets — with no consistent naming convention or indexing. When a specific invoice needs to be retrieved for a vendor query, dispute resolution, or audit, **locating the record is a manual search operation** that can take minutes or hours per request. At scale, this represents a significant operational drag on the finance function.

4 — Delayed Processing Cycles

Manual review and data entry create a processing bottleneck at every stage of the AP workflow. Invoices queue for human attention before they can be entered, approved, or scheduled for payment. These delays **extend payment cycles, risk late payment penalties, and damage vendor relationships** — particularly with suppliers who operate on tight cash flow margins.

5 — Inaccessible Historical Records

Finance teams cannot query, aggregate, or report on historical invoice data without first digitising it. Spend analysis, vendor benchmarking, audit preparation, and budget forecasting all depend on having structured, queryable records — but as long as invoices remain in unstructured formats, **none of that reporting capability is accessible** without significant manual effort upstream.

Cost of Inaction

Each of these pain points compounds the others. Manual entry creates unvalidated data; unvalidated data creates errors; errors slow approval cycles;

slow cycles damage vendor relationships; and inaccessible records prevent organisations from even quantifying the scope of the problem.

SECTION 3

Why Traditional OCR Is Not Enough

Deploying a standard OCR engine to automate invoice processing is a commonly attempted — and commonly failed — approach. The limitations of traditional OCR are well-documented, and for AP invoice processing specifically, they are disqualifying without an additional intelligence layer.

Key Limitations of Traditional OCR

Limitation	Impact on AP Invoice Processing
Image Quality Sensitivity	Poor scan quality, low resolution, skewed documents, or faded print significantly degrade character recognition accuracy. A single unclear digit in an invoice total produces an incorrect financial record.
Complex Document Layouts	Invoices vary enormously in structure across vendors. Multi-column layouts, irregular formatting, nested line item tables, and non-standard header positioning cause standard OCR to misread or misorder extracted text.
Non-Text Element Interference	Vendor logos, company stamps, authorisation signatures, and security watermarks overlap with or sit adjacent to text. Traditional OCR cannot distinguish decorative elements from meaningful content, resulting in

Limitation	Impact on AP Invoice Processing
	corrupted extraction.
No Interpretation Layer	Traditional OCR digitises characters — it does not understand them. It cannot determine whether a string of text is a vendor name, a line item description, a PO reference, or a footer note. The output is an undifferentiated wall of recognised characters with no semantic structure.
Special Character and Format Loss	Currency symbols, decimal separators, date formats, and thousands separators are frequently misread or dropped entirely. A figure like \$1,234.50 may be extracted as 1 23450 or omitted — rendering the data financially unusable.

The Net Result

These limitations do not merely reduce the quality of extracted data — they create **workflow discontinuities**. When OCR output cannot be trusted, downstream systems reject records, exceptions pile up, and human reviewers are pulled in to correct machine errors rather than focus on genuine exceptions. In many deployments, the correction burden of a poorly implemented OCR system rivals or exceeds the cost of the manual process it was intended to replace.

The Core Gap

Traditional OCR converts an image to text. What AP invoice processing requires is a system that converts an image into **structured, validated, semantically meaningful financial data**. These are fundamentally different tasks, and traditional OCR was not designed for the latter.

How We Intend to Solve This

Each pain point and technical limitation identified above is mapped to a specific component of the pipeline. The architecture is deliberate: no single component carries the full burden of accuracy, and the layers are designed to compensate for each other's inherent constraints.

Solution-to-Problem Mapping

Component	Problems Addressed	How It Solves Them
GCP Document AI	Image quality sensitivity; complex layouts; non-text element interference	Google's Document AI is trained on large corpora of real-world document types, including invoices. It is far more robust to layout variation, noise, and image degradation than commodity OCR engines. It understands document structure contextually, not just character-by-character.
LLM Extraction Layer	No interpretation layer; special character loss; manual data entry	Sits directly on top of Document AI output. The LLM interprets, classifies, and structures the extracted text, returning a clean JSON schema: vendor, invoice_date, total, and line_items[]. This closes the semantic gap that traditional OCR leaves entirely unaddressed.
Automated Validation Engine	No validation layer; manual cross-checking errors	After extraction, the engine programmatically

Component	Problems Addressed	How It Solves Them
		sums all <code>line_item.amount</code> values and compares the result to the declared <code>total</code> field. Any arithmetic discrepancy is immediately flagged — eliminating the need for manual cross-checking and catching errors at the point of ingestion rather than at the point of payment.
FastAPI REST Endpoint	Delayed processing cycles; inaccessible records; poor searchability	Exposes the entire pipeline as a standard REST API. ERP systems, accounting platforms, and downstream workflows can submit invoices and receive structured JSON in a single request — enabling real-time integration without human intermediation and making every processed invoice immediately queryable.
Streamlit UI	Exception management; finance team oversight; flagged discrepancies	Provides a human-in-the-loop review interface for exceptions, flagged discrepancies, and quality assurance cases. Finance teams retain full visibility and control without being required to handle routine invoice processing — their attention is directed only where it is genuinely needed.

End-to-End Flow Summary

1. **Invoice Submitted** — An invoice (PDF or image format) is submitted via the FastAPI endpoint or uploaded through the Streamlit UI.
2. **Document AI Parsing** — GCP Document AI performs layout-aware OCR, producing structured document text with high tolerance for image noise and format variation.
3. **LLM Extraction** — The LLM layer interprets the Document AI output and returns a structured JSON object conforming to the defined schema.
4. **Automated Validation** — The validation engine sums all line item amounts and compares against the declared total. Discrepancies are flagged; clean records are passed forward.
5. **ERP Integration or Human Review** — Clean records are pushed downstream via API. Flagged records are routed to the Streamlit UI for finance team review and resolution.

Design Principle

The pipeline is designed so that the **common case requires zero human involvement** — invoices that parse cleanly and validate correctly flow straight through to downstream systems. Human reviewers are engaged only for genuine exceptions, maximising both automation benefit and human oversight quality.

SECTION 5 — Next Steps: Live-In Documentation & Self-Auditing Layer

What Is Being Added — Not Replaced

The LLM is the core engine of this project. Its primary responsibility is invoice field extraction: it receives raw output from GCP Document AI, interprets it, and returns a clean, structured JSON payload. That does not change.

What is being added is an additional capability layer that runs within the same pipeline. This layer is a **live-in documentation system** that allows the LLM to also audit, validate, and explain the state of the codebase against its own living documentation. It does not replace the extraction function — it sits alongside it, callable as part of the same pipeline run, adding a self-auditing dimension to a system that is already doing the heavy lifting of invoice intelligence.

Why This Is Being Implemented In The Same Project

This is one project, built one step at a time in real time. The live-in documentation layer is the **finishing stage** of the Automated AP Invoice Processor — not a separate tool, not a future project. It is designed to sit inside the same codebase and be invoked as part of the same pipeline execution.

The reason it is implemented here rather than bolted on later is intentional. Documentation that lives inside the project — versioned alongside the code, maintained in the same repository, readable by the same LLM that runs the pipeline — is always current. It cannot drift. When a module changes, its documentation changes with it. When the LLM audits the code, it reads the living source of truth, not a stale wiki page written at a different point in the project's life.

What The LLM Now Does — Dual Responsibility

In the completed project, the LLM carries two distinct responsibilities:

Responsibility	Trigger	Input	Output
PRIMARY — Invoice Extraction	Every pipeline run	Raw token output from GCP Document AI	Structured JSON: <code>vendor,</code> <code>invoice_date,</code> <code>total,</code> <code>line_items[]</code>
SECONDARY — Documentation Auditing (finishing stage)	Invoked alongside extraction, per module	Living documentation file for the target module + the corresponding code file	Alignment confirmation (silent pass) or a detailed, doc-sourced explanation of what is wrong and how to fix it

The secondary responsibility is not a passive log. When a discrepancy is found, the LLM does not emit a generic error code — it explains the issue in plain language, drawing directly from the living documentation to describe what the code should be doing, what it is currently doing, and what the resolution path is.

How It Works In Practice

When a pipeline task is run, the doc-checker is invoked alongside the main extraction task. The sequence is as follows:

6. **Module identification** — The pipeline identifies which module is being executed (e.g., the extraction layer, the validation engine, the API handler).
7. **Documentation read** — The LLM reads the living documentation file for that module. This is the versioned, in-repo source of truth describing what the module is supposed to do, what its inputs and outputs are, and what its constraints are.
8. **Code read** — The LLM reads the corresponding code file for the same module.
9. **Cross-reference** — The LLM compares the two. If everything is aligned — the code does what the documentation says it does — the checker passes silently and the pipeline continues.

10. **Discrepancy surfacing** — If there is a divergence — a missing field, a broken function signature, a deprecated method still in use, a behaviour that contradicts the documented contract — the LLM surfaces a detailed explanation. This explanation is drawn directly from the living docs: it states what the documentation specifies, what the code currently does, and what the engineer needs to address.
11. **Engineer review** — The engineer (Makky) receives the output and decides how to act. The LLM does not auto-fix anything. Its role is to surface, explain, and inform — human judgement makes the call.

Why This Makes The Project Hiring-Ready

Most portfolio projects demonstrate that something works. They show that a candidate can write code that produces an output. This project demonstrates something more specific and more valuable.

By implementing a live-in documentation and self-auditing layer, this project demonstrates that the engineer understands:

12. **Maintainability** — code written to be understood and updated, not just executed.
13. **Documentation discipline** — documentation treated as a living artefact, not an afterthought written once and left to decay.
14. **Production-grade error handling** — discrepancies surfaced with context and resolution guidance, not silent failures or raw stack traces.
15. **LLM integration as a workflow tool** — the model used as an active participant in the development and maintenance lifecycle, not just as an output generator called once and forgotten.

That combination — working software, intelligent automation, self-auditing infrastructure, and documented discipline — is precisely what separates a junior portfolio from a senior engineering artefact. It signals that the engineer building this project thinks about what happens after the first deployment, not just whether the demo runs.

